Portfolio Stream Feed

The Order update stream feed communicates all order updates when connected over the websocket. No request object is required for this. The communication will start immediately after the connection is successfully established.

Integration involves connecting to the provided endpoint using the `wss:` protocol through a WebSocket client, which is configured to automatically redirect to the authorized WebSocket endpoint upon authentication. Therefore, the client's configuration must be adjusted to support this automatic redirection. For example, in a Node.js client, you should enable the `followRedirects` setting to facilitate seamless handling of redirection.

INFO

The order update WebSocket ensures that you receive updates on your order regardless of the platform used to place it. Whether you place an order through the Upstox mobile app or via the web application, you will still receive the order updates through the API WebSocket, and the same applies in reverse.

Header Parameters

Name	Required	Type	Description
Authorization	true	string	Requires the format `Bearer access_token` where `access_token` is obtained from the Token API.
Accept	true	string	Defines the content format the client expects, which should be set to `*/*`.

Query Parameters

Name	Required	Type	Description
update_types	false	string	Identifiers, separated by commas, specify the types of updates to receive. By default, updates on orders are transmitted via the socket. To subscribe to updates on orders, holdings, and

Name	Required	Type	Description
			positions, use this parameter by entering the desired values, separated by commas. Possible values: `order`, `gtt_order`, `position`, `holding` Sample URL with update_types: `wss://api.upstox.com/v2/feed/portfolio-stream-feed?update_types=order%2Cposition%2Cholding`

Responses 302

This API does not provide a typical JSON response. Instead, upon successful authentication, it automatically redirects the client to the appropriate websocket endpoint where order updates can be received in real-time. Users are expected to handle data streams as per the websocket protocol once the redirection is complete.

Following is a sample of an order update message received on the socket.

Message Structure

Order GTT Order Holding Position

```
{
  "update_type": "order",
  "user_id": "******",
  "userId": "******",
  "exchange": "NSE",
  "instrument_token": "NSE_EQ|INE848E01016",
  "instrument_key": "NSE_EQ|INE848E01016",
  "trading_symbol": "NHPC-EQ",
  "tradingsymbol": "NHPC-EQ",
  "product": "D",
  "order_type": "MARKET",
  "average_price": 0,
  "price": 0,
  "trigger_price": 0,
```

```
  "quantity": 1,
  "disclosed_quantity": 0,
  "pending_quantity": 1,
  "transaction_type": "BUY",
  "order_ref_id": "57744821658411",
  "exchange_order_id": "",
  "parent_order_id": null,
  "validity": "DAY",
  "status": "put order req received",
  "is_amo": false,
  "variety": "SIMPLE",
  "tag": null,
  "exchange_timestamp": null,
  "status_message": "",
  "order_id": "240221025997024",
  "order_request_id": "1",
  "order_timestamp": "2024-02-21 14:40:02",
  "filled_quantity": 0,
  "guid": null,
  "placed_by": "******",
  "status_message_raw": null
}
```


Name	Type	Description
update_type	string	Signifies the type of update received. Possible values: `order`, `gtt_order`, `position`, `holding`
exchange	string	Exchange to which the order is associated. Valid exchanges can be found in the Exchange Appendix
product	string	Signifies if the order was either Intraday, Delivery or MTF Possible values: `I`, `D`, `MTF`.
price	float	Price at which the order was placed
quantity	int32	Quantity with which the order was placed
status	string	Indicates the current status of the order. Valid order statuses can be found in the Order Status

Name	Type	Description
		Appendix
tag	string	Tag to uniquely identify an order
instrument_token	string	Key of the instrument. For the regex pattern applicable to this field, see the Field Pattern Appendix.
instrument_key	string	Key of the instrument. For the regex pattern applicable to this field, see the Field Pattern Appendix.
placed_by	string	Uniquely identifies the user (commonly referred as UCC)
trading_symbol	string	Shows the trading symbol of the instrument
order_type	string	Type of order. It can be one of the following MARKET refers to market order LIMIT refers to Limit Order SL refers to Stop Loss Limit SL-M refers to Stop Loss Market. Possible values: `MARKET`, `LIMIT`, `SL`, `SL-M`.
validity	string	It can be one of the following - DAY(default), IOC. Possible values: `DAY`, `IOC`.
trigger_price	float	If the order was a stop loss order then the trigger price set is mentioned here
disclosed_quantity	int32	The quantity that should be disclosed in the market depth
transaction_type	string	Indicates whether its a buy or sell order. Possible values: `BUY`, `SELL`.
average_price	float	Average price at which the qty got traded

Name	Type	Description
filled_quantity	int32	The total quantity traded from this particular order
pending_quantity	int32	Pending quantity to be filled
status_message	string	Indicates the reason when any order is rejected, not modified or cancelled
status_message_raw	string	Description of the order's status as received from RMS
exchange_order_id	string	Unique order ID assigned by the exchange for the order placed
parent_order_id	string	In case the order is part of the second of a CO, the parent order ID is indicated here
order_id	string	Unique order ID assigned internally for the order placed
variety	string	Order complexity
order_timestamp	string	User readable timestamp at which the order was placed
exchange_timestamp	string	User readable time at which the order was placed or updated
is_amo	boolean	Signifies if the order is an After Market Order
order_request_id	string	Apart from 1st order it shows the count of how many requests were sent
order_ref_id	string	Uniquely identifies an order for internal usage.

HEARTBEAT

If there is no data to stream over an open WebSocket connection, the API automatically sends a standard `ping` frame periodically to maintain the connection's aliveness. Most standard WebSocket client libraries across various programming languages handle this automatically by responding with a `pong` frame, requiring no manual intervention.

NOTICE OF DEPRECATION

The lowercase field (`tradingsymbol`) is deprecated and will be removed in future versions. Use the snake_case versions for consistency.

WSS /feed/portfolio-stream-feed

Request

LIVE URL
wss://api.upstox.com/v2

+ Show optional parameters